

THECC Fair Value Gaps PRO — Developer Guide

THECC Institutional Trader Academy™

 Course Metadata

Field	Value
Academy	THECC Institutional Trader Academy™
Course	Fair Value Gaps PRO
Module	Engineering
Lesson	Developer Guide
Difficulty	Advanced
Estimated Time	40 minutes
Prerequisites	Pine Script v6 working knowledge
Indicator Version	v2.1
Document Version	v2.1.0
Last Updated	2026-09-20
Instructor	Dale Batise
Status	Content Complete — Certification Pending

Document control:

Field	Value
Document ID	THECC-DOC-FVGPRO-DG
Document type	Developer Guide
Indicator	THECC Fair Value Gaps PRO · Registry ID pending

Field	Value
Version	v2.1.0
Build	2.1.0+2026-09-20
Release date	2026-09-20
Compatible indicator versions	FVG PRO v2.1.x
Academy Doc Standard	v1.0
Edition	Internal Master
Platform	TradingView · Pine Script v6
Copyright	© THE CONSISTENCY COLLECTIVE

Consistency Creates Freedom.

1 · Source and licensing

Item	Value
Source file	pine/thecc-fvg-pro-v2.pine
Lines	~1,040
Pine version	v6
Declaration	overlay=true , 500 boxes / 500 labels / 500 lines, max_bars_back=5000
Compile status	Not green-checked

⚠️ Licensing. The source header carries third-party attribution required by its open-source licence. That attribution stays in the `.pine` comments and must never be removed. It must equally never appear in student-facing documentation, the settings panel or any on-chart text — Academy Documentation Standard v1.0 §6a and §7.

2 · Execution order

The script is a single pass per bar, in this order. Order matters: helpers and state must be declared before use, and `alertcondition()` must sit at global scope.

1 constants + indicator()	maxBoxesCount must stay const
2 inputs	TF mode → FVG → Zone Engine → Style → Skull → Market Structure → EQH/EQL → Sweeps → Alerts
3 shared helpers	n = bar_index, sizeOf, lineStyleOf, labStyleOf
4 types	FVG, ZZ, MSSState
5 FVG helpers	createFVG ... combineFVGsFunc
6 TF helpers	tfToLabel, tfToSeconds, safeTF
7 detectFVG()	pure detection, called inside request.security
8 processSlot() / renderSlot()	lifecycle and drawing
9 TF resolution + storage	5 slots: chart/fix + 4 multi
10 request.security × 5	lookahead_off
11 processing	per active slot
12 render	barstate.islast only
13 market structure	zig-zag, MSS/BOS, per-bar signal flags
14 equal highs/low	pivot pairs within tolerance
15 liquidity sweeps	lookback extremes, label array + cap
16 alerts	alertcondition() ×17, then alert() calls

3 · Types

Type	Purpose	Notes
FVG	One zone: bounds, volume split, timestamps, timeframe tag, and its four boxes and two lines	endTime is na while live
ZZ	Fixed 50-slot zig-zag of swing direction, bar index and price	Rolled with unshift / pop
MSSState	Structure direction plus eight arrays of lines and labels	Direction is 1, -1 or 0

4 · Detection contract

`detectFVG()` returns `[bullValid, bearValid, hi, lo, totalVol, highVol, lowVol, startTime]` and is deliberately **pure** — no drawing, no state — so it can be called inside `request.security` for each

timeframe slot.

Conditions, in the order they are applied:

1. **Bar-type gate** — `Same Type` requires three same-direction bars; `All` accepts any.
2. **Quality filter** — Average Range: combined three-bar body size $\times$ sensitivity $>$ $ATR \div 1.5$. Volume Threshold: `sma(volume,5) > sma(volume,15)  $\times$  threshold`.
3. **Jump check** — the larger of the two bar-to-bar open/close jumps must be $\leq$ ATR, unless Allow Gaps is on.
4. **Geometry + middle-bar close** — bullish: `low > high[2]` and `close[1] > high[2]`; bearish: `high < low[2]` and `close[1] < low[2]`.
5. **Size check** — `gapSize  $\times$  sensitivity > ATR`, unless the sensitivity checkbox is off.

💡 Sensitivity multiplies the *measured* value, not the threshold, which is why a higher number is more permissive. Any future rewording of that control must preserve this, or every existing user's chart changes meaning.

5 · Lifecycle

`processSlot()` runs only when `bar_index > last_bar_index - maxDistanceToLastBar` **and** `barstate.isconfirmed`. This is what makes zones non-repainting, and it is the single most important line in the file.

Per bar: create (deduplicated by `startTime`), cap the list at `showLastXFVGs`, update `lastTouched`, invalidate by wick or body close, then remove anything failing `isFVGValid()`.

`renderSlot()` runs only on `barstate.islast`: merge overlapping same-direction zones, extend the newest `extendLastXFVGsCount` live zones, delete and redraw. Appearance can therefore change on the last bar even though the underlying zones do not.

6 · Drawing cost

Element	Boxes	Lines	Labels
Zone (either mode)	4	2	0
MSS / BOS event	0	1	1
Equal-level shelf	0	1	1
Liquidity sweep	0	0	1

Ceiling is 500 of each, so **boxes bind first: $500 \div 4 \approx 125$ zones across all slots**. Skull Mode still allocates four boxes — one is an invisible placeholder that keeps `safeDeleteFVG()` total.

⚠ Any feature that adds per-zone drawings multiplies against 125. Adding a single extra box per zone drops capacity to 100.

7 · Structure, levels and sweeps

- **Zig-zag** — `ta.pivohigh(high, msLen, 1)` and `ta.pivotlow(low, msLen, 1)`; one right bar, so confirmation lags by one bar. A new pivot in the same direction updates the head rather than pushing a new entry.
- **MSS / BOS** — evaluated against `aZZ` indices 1 and 2. MSS flips `msState.dir`; BOS requires the direction already set and is suppressed when the level equals the last drawn line's `y2`.
- **Equal levels** — `ta.pivohigh(high, eqLen, eqLen)`: confirmation lags `eqLen` bars on both sides. Equality is `|a - b| ≤ lastPivot × tolerance ÷ 100`.
- **Sweeps** — `ta.highest(high[1], lookback)` excludes the current bar. Labels are pushed to `liqLabels` and popped past `maxLiqLabels`.

8 · Alerts

Seventeen `alertcondition()` calls at global scope, plus six `alert()` calls with `alert.freq_once_per_bar`. Per-bar boolean flags (`sigMssBull`, `sigBosBear`, `sigBuySweep`, ...) are set inside the calculation blocks and consumed in the alert section.

`okBar = not alertOnClose or barstate.isconfirmed` gates every dynamic alert. Because Pine rolls back `var` state on each realtime tick, the confirmed-tick evaluation is correct without extra guarding.

⚠ BOS detection sits inside the `iBOS` branch, so **Show BOS off means no BOS alert**. If that coupling is ever unwanted, the flag assignment must move out of the drawing branch — a deliberate change, not a tidy-up.

9 · Extending safely

Change	Watch
New per-zone drawing	Recompute the 125-zone budget
New alert	Add the <code>alertcondition()</code> at global scope; local scope will not compile

Change	Watch
New input	Keep the default equal to current behavior, or it is a breaking change
Higher-timeframe structure	Would need <code>request.security</code> plus bar-index translation — drawings currently anchor to chart bars
Renaming a control	Users' saved settings key off input order and type; renaming resets nothing but re-labels meaning
Touching <code>detectFVG()</code>	Every documented number in the suite derives from it; regenerate the docs

10 · Test checklist before release

1. Compile: 0 errors. Record the green-check date.
2. Auto / Fix / Multi-Fix with 1, 3 and 5 active slots.
3. A symbol with no volume feed — text must fall back to `FVG` .
4. Both invalidation methods against a wick-pierced zone.
5. Sensitivity at all four steps: monotonic zone counts.
6. Structure with `msClearOnShift` on and off.
7. Every alert: named conditions and the dynamic route, bar-close gating on and off.
8. Budget stress: four timeframes at 125 zones, confirm the documented failure signature.
9. Bar Replay: confirm zones appear only on confirmed bars.

Revision History

Version	Date	Author	Change
v2.1.0	2026-09-20	Dale Batise	First Developer Guide for Fair Value Gaps PRO v2.1: execution order, types, detection contract, lifecycle, drawing costs, structure internals, alert wiring, extension risks and the pre-release test checklist.

Educational material only. Not financial advice, not a signal service, and not a claim about future results.

